

Connect a GitHub Repo with AWS

Introducing Today's Project!

In this project, I will demonstrate how to set up Git, connect my web application to a GitHub repository, and manage code changes using version control in a real development workflow. I will link my local project to GitHub, push updates, and ensure my code is stored, tracked, and accessible in the cloud. I'm doing this project to learn how Git and GitHub work in a CI/CD pipeline, how teams collaborate on shared codebases, and how to track and manage changes efficiently. This is essential because without version control, managing updates, collaboration, and deployments becomes chaotic and unreliable.

Key tools and concepts

Services I used were Amazon EC2 (to host my cloud-based development environment), VS Code with the Remote - SSH extension (to connect to and edit files on my EC2 instance), and GitHub (to store, version, and manage my web application's source code remotely). I also used Apache Maven to generate the Java web app structure and Amazon Corretto 8 as my Java runtime environment. Key concepts I learnt include version control using Git, the difference between a local repository and a remote repository, how to initialize a Git repo with `git init`, staging changes with `git add`, committing snapshots with `git commit`, and pushing changes to a remote origin with `git push`. I also learnt about GitHub personal access tokens as a secure alternative to password authentication, how to configure a Git identity using `git config`, and how to write and format a professional README file using Markdown.

Project reflection

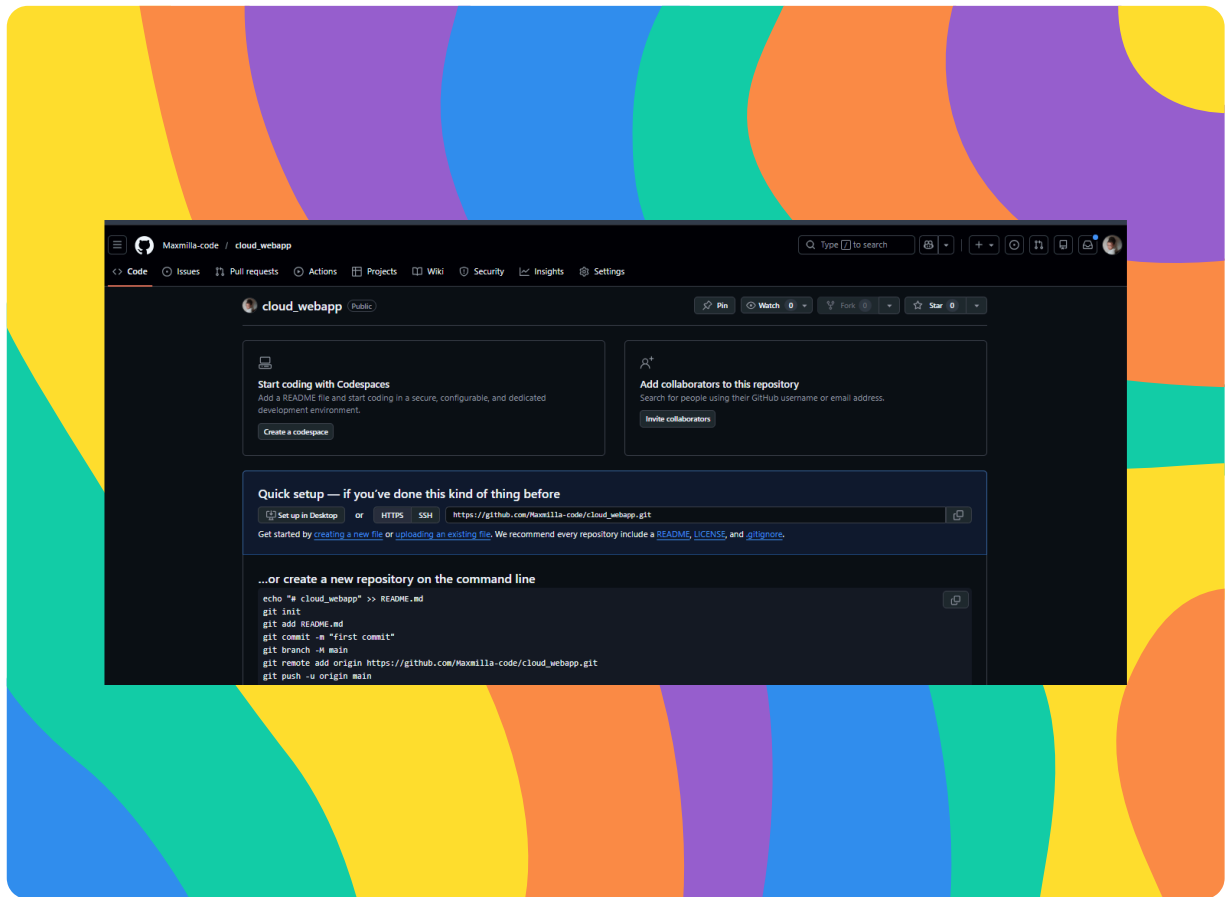
This project took me approximately 2 hours to complete everything from start to finish. The most challenging part was ensuring all steps were followed carefully and in the correct order — and one important disclaimer worth noting: when pushing code to GitHub, always make sure you have the correct project folder open in VS Code. Pushing from the wrong directory can send unintended files to your repository, so double-checking your working directory with `pwd` before running any Git commands is a simple but critical habit to build. It was most rewarding to see my web app files appear live on GitHub after running `git push` for the first time watching the local and remote repositories sync up in real time made the whole version control workflow click into place.

I did this project because I wanted to gain more hands-on knowledge in the DevOps space, and specifically to understand how Git and GitHub work within an EC2 cloud environment. Getting to experience initializing a repository, making commits, authenticating with a personal access token, and pushing real code changes to GitHub from inside a remote server was incredibly insightful. Beyond the technical steps, it was the overall flow of the project that made it an awesome experience — everything connected logically, from setting up the instance to seeing live changes reflected in my GitHub repository.

This project is part two of a series of DevOps projects where I'm building a CI/CD pipeline! I'll be working on the next project very soon — it's Part 3: Secure Packages with AWS CodeArtifact, where I'll be exploring how to store and manage my web app's packages and dependencies securely in the cloud. I'll be uploading everything I do as part of my ongoing documentation, so stay tuned for that!

Git and GitHub

GitHub is a cloud-based platform that stores and manages code using Git, allowing developers to track changes, access projects from anywhere, and collaborate with others more easily. I'm using GitHub in this project to store my web application code remotely, track all changes in a structured way, and enable collaboration and integration with a CI/CD workflow.



My local repository

A Git repository is a storage location where your project's code and its entire history of changes are tracked, allowing you to manage versions, collaborate with others, and revert to previous states when needed.

Git init is a command that initializes a new local Git repository in a directory, setting up all the necessary files for Git to start tracking changes in that folder. I ran git init in my nextwork-web-project folder on the EC2 instance, which allowed Git to start monitoring all file changes for version control.

A branch in Git is a separate line of development within a repository that lets you work on changes independently from the main version of your code. After running git init, the response from the terminal mentioned the default branch name, usually master or main, which is the main branch where all changes are saved unless you create additional branches to experiment safely.

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS

• [ec2-user@ip-172-31-38-103 ~]$ git --version

git version 2.50.1
• [ec2-user@ip-172-31-38-103 ~]$ git init
hint: Using 'master' as the name for the initial branch. This default branch name
hint: is subject to change. To configure the initial branch name to use in all
hint: of your new repositories, which will suppress this warning, call:
hint:
hint:   git config --global init.defaultBranch <name>
hint:
hint: Names commonly chosen instead of 'master' are 'main', 'trunk' and
hint: 'development'. The just-created branch can be renamed via this command:
hint:
hint:   git branch -m <name>
hint:
hint: Disable this message with "git config set advice.defaultBranchName false"
Initialized empty Git repository in /home/ec2-user/.git/
○ [ec2-user@ip-172-31-38-103 ~]$
```

To push local changes to GitHub, I ran three commands

`git add`

The first command I ran was `git remote add origin <your-repo-URL>`. This command connects my local project folder on the EC2 instance to the GitHub repository by telling Git where the remote repository is located. Think of `origin` as a bookmark for my GitHub repo so I don't have to type the full URL every time I want to push changes. A staging area is a place in Git where changes to files are collected and prepared before being committed. It lets me review all edits together and decide exactly which modifications should be saved in the next version of the project.

`git commit`

The second command I ran was `git add ..` This command stages all the files in my project folder, marking them to be included in the next commit. Using `-m` is not part of this command; it's used later with `git commit` to include a message describing the changes being saved.

`git push`

The third command I ran was `git commit -m "Updated index.jsp with new content"`. This command saves all the staged changes as a new snapshot in the project's version history.

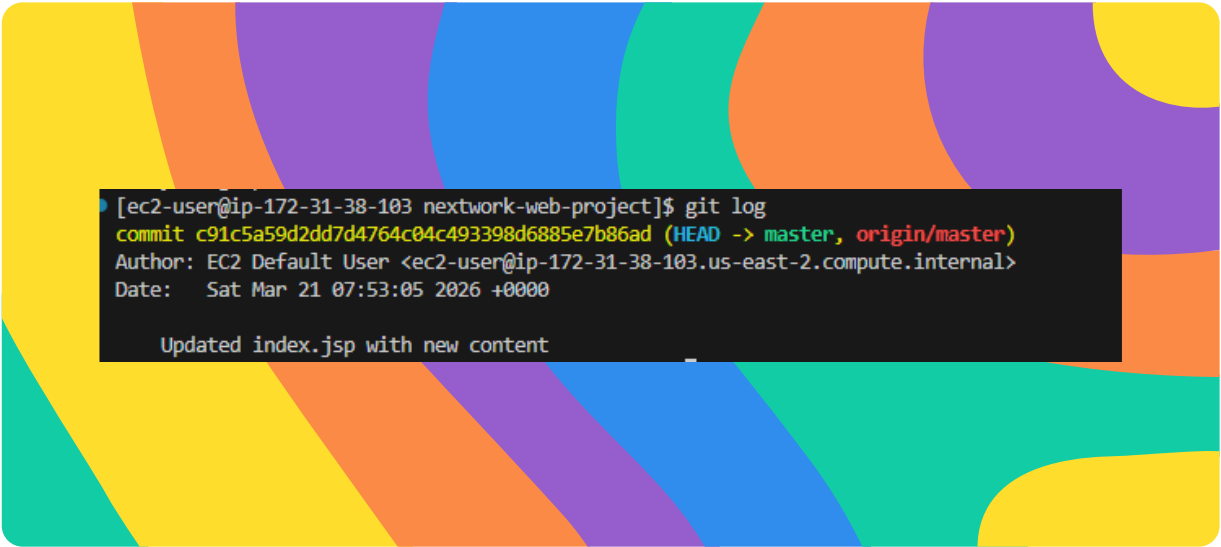
Using -u is not part of this command; it's used later with git push to set the upstream branch for future pushes.

Authentication

Local Git identity

Git needs my name and email because it uses them to identify the author of each commit. This information allows Git to track who made specific changes, maintain a clear version history, and ensure accountability when multiple people are collaborating on the same repository. Without it, commits would lack proper attribution, making it harder to review changes, debug issues, or manage contributions.

Running git log showed me that all my commits were recorded in order, including the commit messages, timestamps, and author information. It confirmed that Git successfully tracked my changes, who made them, and when they were made, giving me a complete history of my project's development and ensuring transparency and traceability for every modification.

A terminal window with a black background and white text, overlaid on a colorful abstract background of yellow, orange, purple, blue, and teal shapes. The terminal shows the output of a 'git log' command. The first line is a prompt: '[ec2-user@ip-172-31-38-103 nextwork-web-project]\$'. The second line is the output: 'commit c91c5a59d2dd7d4764c04c493398d6885e7b86ad (HEAD -> master, origin/master)'. The third line is 'Author: EC2 Default User <ec2-user@ip-172-31-38-103.us-east-2.compute.internal>'. The fourth line is 'Date: Sat Mar 21 07:53:05 2026 +0000'. Below the log output, there is a commit message: 'Updated index.jsp with new content'.

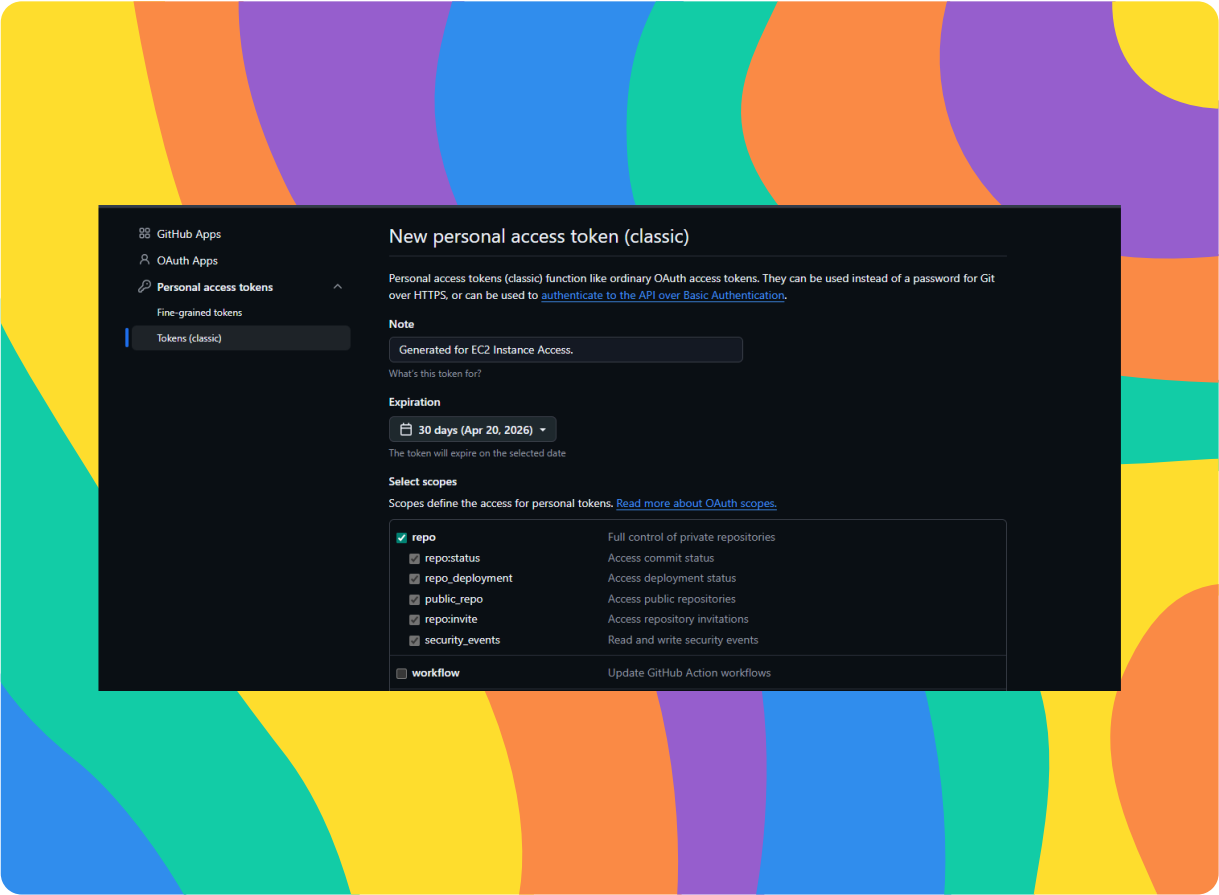
```
[ec2-user@ip-172-31-38-103 nextwork-web-project]$ git log
commit c91c5a59d2dd7d4764c04c493398d6885e7b86ad (HEAD -> master, origin/master)
Author: EC2 Default User <ec2-user@ip-172-31-38-103.us-east-2.compute.internal>
Date: Sat Mar 21 07:53:05 2026 +0000

    Updated index.jsp with new content
```

GitHub tokens

A GitHub token is a secure, randomly generated string that acts like a password for authentication when accessing GitHub repositories. I'm using one in this project because GitHub no longer allows password-based authentication, and the token provides a safer way to verify my identity when pushing or interacting with my repository.

I could set up a GitHub token by going to my GitHub profile settings, navigating to Developer settings, selecting Personal access tokens, then Tokens (classic), and generating a new token. I gave it a descriptive name, set an expiration period, selected the repo scope for permissions, generated the token, and copied it immediately to store it securely.



Making changes again

I wanted to see Git working in action, so I updated the `index.jsp` file by adding a new line of HTML content to my web application. I couldn't see the changes in my GitHub repo initially because saving the file in VS Code only updated my local repository on the EC2 instance, and those changes are not automatically synced to GitHub without using Git commands to push them.

I finally saw the changes in my GitHub repo after staging the updates with `git add .`, reviewing them using `git diff --staged`, committing them with `git commit -m "Add new line to index.jsp"`, and then pushing them to GitHub using `git push`.

cloud_webapp / src / main / webapp / index.jsp



Maxmilla-code Add new line to index.jsp

c13e4c2 · 1 minute ago

History

Code

Blame

11 lines (8 loc) · 301 Bytes



Raw



```
1 <html>
2 <body>
3
4 <h2>Hello maxmilla</h2>
5 <p>I am writing this line using nano instead of an IDE.</p>
6 <p>Im writting this line of code using nano instead of my normal IDE</p>
7 <p>If you see this line in Github, that means your latest changes are getting pushed to your cloud repo :o</p>
8
9
10 </body>
11 </html>
```

Setting up a README file

As a finishing touch to my GitHub repository, I added a README file, which is a document that introduces and explains my project, including what it does, how it is set up, and how it can be used. It helps anyone viewing the repository quickly understand the purpose and structure of the project, making it more professional and easier to work with. I added a README file by running the command ``touch README.md`` in my terminal, which created a new blank file that I can now edit and fill with relevant project documentation.

My README is written in Markdown because it allows me to format and structure my project documentation in a clean, readable way without needing complex tools, making it ideal for displaying content on platforms like GitHub. Special characters can help you format text in Markdown, such as ``#`` for headers, ``**`` for bold text, ``*`` for italics, ``-`` or ``1.`` for lists, ``[text](URL)`` for links, and triple backticks ````` for creating code blocks.

My README file has 6 sections that outline the purpose of the project, the technologies used, how to set it up, the key steps involved in building the application, reflections on challenges and learning outcomes, and contact information for collaboration or questions.

Java Web App Deployment with AWS CI/CD

Welcome to this project combining Java web app development and AWS CI/CD tools!

Table of Contents

- [Introduction](#)
- [Technologies](#)
- [Setup](#)
- [Contact](#)
- [Conclusion](#)

Introduction

This project is used for an introduction to creating and deploying a Java-based web app using AWS, especially their